

Yes. For this project, I would **not put every feature into one giant table**. The clean architecture is to keep the raw/history data in separate tables and generate a final **node-day / shipment-event feature table** for ML.

For **Frankfurt → India shipment-delay prediction**, this is the complete column design I recommend.

1. route_nodes — master checkpoint table

One row = one checkpoint/location.

Column	Type	Description
node_id	STRING	Unique checkpoint ID
node_name	STRING	Frankfurt, Suez, Mumbai, etc.
node_type	STRING	factory / port / canal / customs / hub / border
country	STRING	Country
region	STRING	Geographic region
latitude	FLOAT	Latitude
longitude	FLOAT	Longitude
timezone	STRING	Local timezone
capacity	FLOAT	Estimated handling capacity
handling_mode	STRING	sea / rail / road / multimodal
is_customs	BOOLEAN	Customs checkpoint
is_transshipment	BOOLEAN	Transshipment hub
is_critical	BOOLEAN	Critical route node
normal_dwell_hours	FLOAT	Typical dwell time
normal_processing_hours	FLOAT	Typical processing duration

2. route_edges — route network

One row = one connection between two checkpoints.

Column	Type
--------	------

edge_id	
---------	--

from_node_id	
--------------	--

to_node_id	
------------	--

mode	
------	--

distance_km	
-------------	--

typical_transit_hours	
-----------------------	--

median_transit_hours	
----------------------	--

p90_transit_hours	
-------------------	--

transit_std_hours	
-------------------	--

historical_delay_rate	
-----------------------	--

historical_mean_delay_hours	
-----------------------------	--

historical_p90_delay_hours	
----------------------------	--

reroute_frequency	
-------------------	--

is_primary_route	
------------------	--

is_alternative_route	
----------------------	--

route_risk_score	
------------------	--

active_from	
-------------	--

active_to	
-----------	--

This becomes your **graph edge information**.

3. shipment_events — actual shipment movement

This is your most important historical table.

One row = shipment reaching/being processed at a checkpoint.

Column	Description
shipment_id	Unique shipment
vehicle_id	Vehicle/container/vessel identifier if available
origin_node_id	Starting node
destination_node_id	Destination
node_id	Current checkpoint
previous_node_id	Previous checkpoint
next_node_id	Next checkpoint
edge_id	Current route segment
event_timestamp	Event time
planned_arrival	Planned arrival
actual_arrival	Actual arrival
planned_departure	Planned departure
actual_departure	Actual departure
planned_dwell_hours	Expected dwell
actual_dwell_hours	Actual dwell
transit_hours	Actual transit
delay_hours	Actual delay
delay_days	Delay in days
delay_flag	Whether delayed
delay_threshold_hours	Threshold
mode	Road/rail/sea
cargo_type	Cargo category
shipment_priority	Priority

Column	Description
route_id	Route identifier

Your main target could be:

delay_flag = 1 if delay_hours > 24 else 0

and:

delay_hours

as the regression target.

4. congestion_history

This is where your **2019 → now ArcGIS backfill** belongs.

One row = node/day.

Column
node_id
date
congestion_raw
queue_length
dwelt_hours
throughput
vessel_count
truck_count
waiting_count
congestion_percentile
rolling_90d_mean
rolling_90d_std
congestion_index

Column

congestion_anomaly

data_quality_flag

source

ingestion_timestamp

Important

Your congestion_index should be calculated from the historical baseline rather than simply treating the raw congestion measurement as the feature.

For example:

congestion_index =

current_congestion / rolling_90d_mean

or whatever exact formula you have defined for your project.

5. weather_history

One row = node/day or node/hour.

Column

node_id

timestamp

temperature

precipitation_mm

wind_speed

wind_gust

wave_height

visibility_km

pressure

storm_flag

Column

heavy_rain_flag

high_wind_flag

rough_sea_flag

weather_severity_score

weather_anomaly_score

source

For maritime nodes, wave_height and wind_speed are especially valuable.

6. conflict_events

One row = one geopolitical/security event.

Column

event_id

event_timestamp

latitude

longitude

country

region

event_type

sub_event_type

fatalities

severity

node_id

distance_to_node_km

distance_to_route_km

Column

affected_corridor

recency_days

recency_decay

conflict_risk_score

source

Then aggregate these into node/day features.

7. disruption_events

This is different from generic conflict data.

It contains events such as:

- Canal blockage
- Port closure
- Strike
- Customs shutdown
- Extreme weather
- War
- Infrastructure failure
- Major accident
- Vessel incident

Column

event_id

node_id

edge_id

event_timestamp

event_end_timestamp

Column

event_type

severity

duration_hours

affected_capacity_pct

route_blocked

rerouting_required

regime_flag

source

description

8. holidays_calendar

Column

date

country

holiday_name

holiday_type

is_public_holiday

is_customs_holiday

is_port_holiday

holiday_load_factor

This lets you model things like:

Indian public holiday → customs/port processing slowdown.

9. node_daily_features — ★ MAIN ML FEATURE TABLE

This is the table I would ultimately feed into your ML models.

One row = one node on one date.

This table combines everything above.

Identity

Column

date

node_id

node_type

country

route_id

Historical delay

Column

delay_mean_7d

delay_mean_30d

delay_mean_90d

delay_mean_365d

delay_std_7d

delay_std_30d

delay_std_90d

delay_p50_30d

delay_p90_30d

delay_p95_30d

delay_rate_7d

delay_rate_30d

delay_rate_90d

Column

delay_rate_365d

delay_ewma

Congestion

Column

congestion_index

congestion_7d_mean

congestion_30d_mean

congestion_90d_mean

congestion_90d_std

congestion_percentile

queue_length

dwel_hours

congestion_anomaly

Weather

Column

temperature

precipitation_mm

wind_speed

wind_gust

wave_height

visibility_km

storm_flag

heavy_rain_flag

high_wind_flag

Column

rough_sea_flag

weather_severity_score

Conflict/geopolitical risk

Column

conflict_count_1d

conflict_count_7d

conflict_count_30d

conflict_count_90d

nearby_conflict_count

conflict_distance_min_km

conflict_severity_mean

conflict_recency_score

conflict_risk_score

Disruption/regime

Column

disruption_flag

disruption_count_7d

disruption_count_30d

route_blocked_flag

rerouting_flag

regime_flag

regime_duration_days

changepoint_score

10. Calendar / temporal features

Add these to node_daily_features.

Column

day_of_week

day_of_month

week_of_year

month

quarter

is_weekend

is_holiday

is_customs_holiday

days_to_holiday

days_from_holiday

sin_day_of_year

cos_day_of_year

sin_month

cos_month

You don't necessarily need all Fourier features. Keep them if validation shows they help.

11. Edge features

For the current node → next node movement:

Column

edge_id

from_node_id

to_node_id

Column

distance_km

mode

typical_transit_hours

median_transit_hours

transit_std_hours

transit_p90_hours

edge_delay_rate

edge_mean_delay

edge_p90_delay

reroute_frequency

alternative_route_count

edge_risk_score

12. Graph features

If you implement the GNN layer, add:

Column

in_degree

out_degree

node_centrality

betweenness_centrality

upstream_delay_mean

upstream_delay_p90

downstream_node_count

distance_to_destination

Column

distance_from_origin

critical_path_flag

The particularly useful ones are:

upstream_delay_mean

upstream_delay_p90

distance_to_destination

critical_path_flag

because they have direct logistics meaning.

13. NLP-derived features

If you implement the LLM layer:

Column

news_event_count

news_risk_score

news_severity

port_bulletin_count

port_bulletin_risk

closure_flag

strike_flag

accident_flag

geopolitical_risk_from_text

weather_risk_from_text

event_confidence

The raw article itself should **not** be passed directly into XGBoost.

Instead:

News

↓

LLM

↓

Structured event

↓

Risk score

↓

ML feature

14. Final target columns

For training, keep the target columns separate from your input features.

Classification

target_delay_flag

Example:

1 = delay > 24 hours

0 = delay <= 24 hours

Regression

target_delay_hours

Quantile forecasting

You don't need separate targets for P50/P80/P90.

The model learns these through quantile loss.

15. Shipment-level final prediction table

After the node model + graph + Monte Carlo layers, you'll have something like:

Column

shipment_id

Column

prediction_timestamp

origin_node_id

destination_node_id

current_node_id

remaining_distance_km

node_delay_probability

predicted_node_delay_hours

route_delay_probability

predicted_total_delay_hours

eta_p10

eta_p50

eta_p80

eta_p90

eta_p95

probability_before_deadline

probability_after_deadline

critical_node_id

critical_node_risk

top_risk_factor_1

top_risk_factor_2

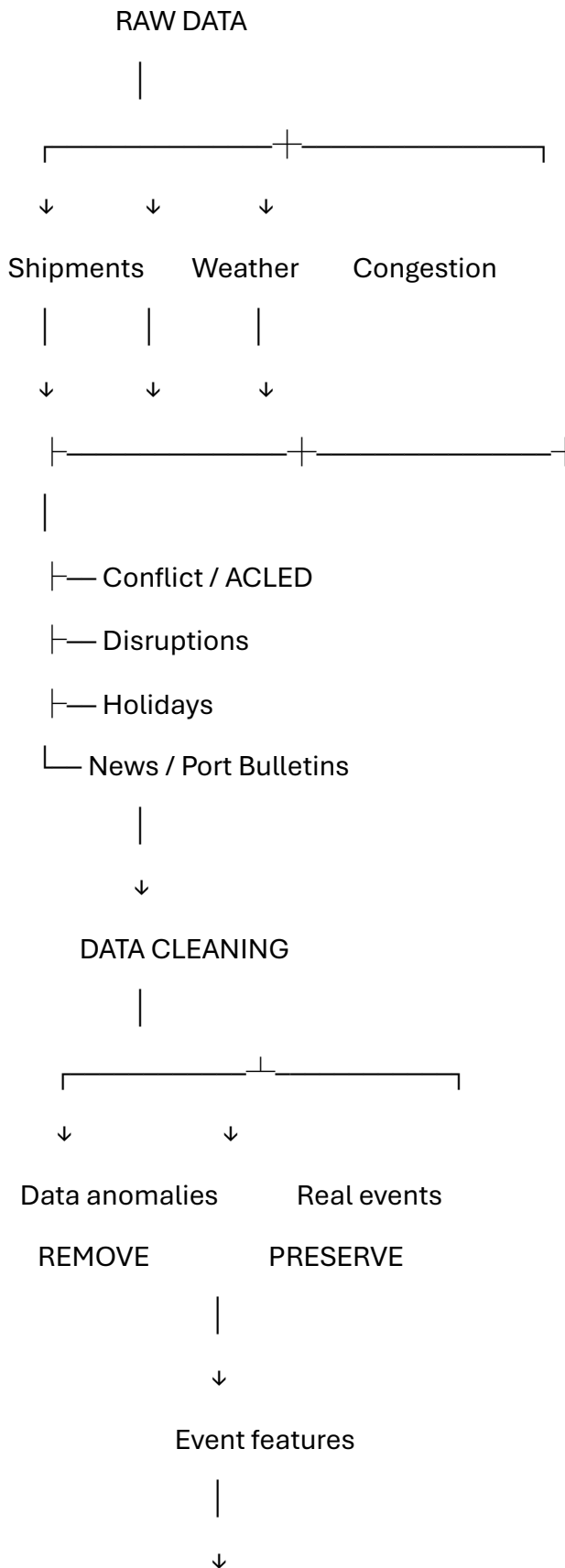
top_risk_factor_3

model_version

This is what your **frontend/dashboard** consumes.

16. The complete architecture

So your data flow becomes:



You **do NOT** need to make all of these columns attributes on every Neo4j node.

That would be a mistake.

Use:

DuckDB/Parquet

For the **large historical analytical data**:

shipment_events

congestion_history

weather_history

conflict_events

node_daily_features

Neo4j

For the **relatively small route graph**:

(Node)

node_id

node_type

location

country

...

(Edge)

edge_id

mode

distance

typical_transit

...

ML feature table

For the **model-ready state of each node at a particular time**:

node_id

date

delay history

congestion

weather

conflict

disruption

calendar

edge features

graph features

target

So conceptually:

Neo4j stores what the network IS.

DuckDB/Parquet stores what HAS HAPPENED to the network over time.

The feature table stores what the model KNOWS at prediction time.

That separation will make the entire system much cleaner and prevent your graph database from becoming a giant historical data warehouse.